

Documentação Técnica do Código — GestSaúde

Controle de Estoque e Almoxarifado

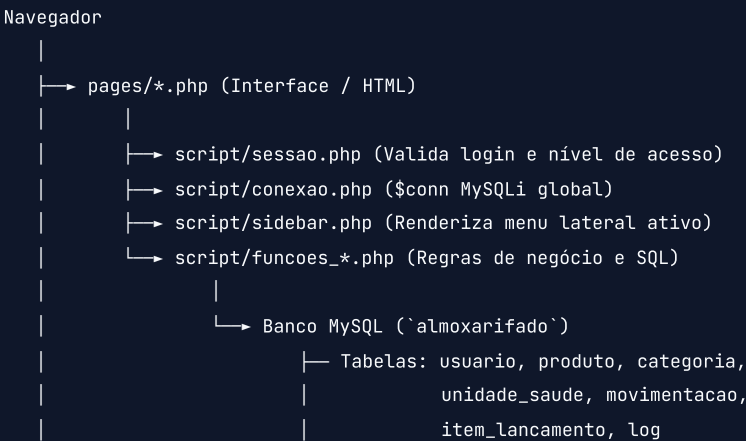
1. Visão Geral do Projeto

O **GestSaúde** é um sistema web em PHP puro e MySQL para controle de estoque de medicamentos e insumos do almoxarifado da Secretaria Municipal de Saúde.

Ele gerencia:

- **Entradas:** Recebimento de materiais no almoxarifado central.
- **Saídas:** Distribuição de materiais para as Unidades Básicas de Saúde (UBS).
- **Estoque e Alertas:** Monitoramento de estoque mínimo e produtos zerados.
- **Auditoria (Logs):** Rastreamento de todas as ações feitas por cada usuário.
- **Usuários:** Controle de acesso por níveis (Administrador, Suporte, Usuario).

2. Mapa de Conexões do Sistema



3. Modelo Lógico do Banco de Dados

O banco de dados se chama **almoxarifado** e adota o padrão relacional com suporte a integridade referencial estrita e transações ACID (mecanismo **InnoDB** obrigatório).

3.1. Esquema Relacional Formal (Notação Lógica)

- **usuario** (id_usuario, nome, email, senha, tipo)
- **categoria** (id_categoria, nome)
- **produto** (id_produto, nome, unidade, estoque, estoque_minimo, #categoria_id)
- #categoria_id referencia categoria(id_categoria)
- **unidade_saude** (id_unidade, nome, endereco, telefone, ativo)
- **movimentacao** (id_movimentacao, tipo, data_hora, observacao, #usuario_id, #unidade_destino_id)
- #usuario_id referencia usuario(id_usuario)
- #unidade_destino_id referencia unidade_saude(id_unidade)
- **item_lancamento** (id_item, quantidade, #movimentacao_id, #produto_id)
- #movimentacao_id referencia movimentacao(id_movimentacao)
- #produto_id referencia produto(id_produto)

- `log` (`id_log`, `data_hora`, `acao`, `descricao`, `#usuario_id`)
- `#usuario_id` referencia `usuario(id_usuario)`

3.2. Dicionário de Dados e Estrutura das Tabelas

Tabela: `usuario` (Contas de Acesso e Permissões)

Campo	Tipo	Nulo	Chave	Descrição / Regra de Negócio
<code>id_usuario</code>	<code>INT</code>	Não	PK	Identificador único incremental do usuário.
<code>nome</code>	<code>VARCHAR(100)</code>	Não	—	Nome completo do usuário.
<code>email</code>	<code>VARCHAR(100)</code>	Não	UK	Email institucional único para login no sistema.
<code>senha</code>	<code>VARCHAR(255)</code>	Não	—	Hash criptográfico gerado via <code>password_hash()</code> (BCrypt).
<code>tipo</code>	<code>ENUM(...)</code>	Não	—	Nível de acesso: <code>'Administrador'</code> , <code>'Suporte'</code> ou <code>'Usuario'</code> .

Tabela: `categoria` (Classificação de Insumos)

Campo	Tipo	Nulo	Chave	Descrição / Regra de Negócio
<code>id_categoria</code>	<code>INT</code>	Não	PK	Identificador único da categoria.
<code>nome</code>	<code>VARCHAR(100)</code>	Não	UK	Nome único da categoria (ex: Medicamentos, Curativos, EPIs).

Tabela: `produto` (Catálogo de Materiais e Estoque Atual)

Campo	Tipo	Nulo	Chave	Descrição / Regra de Negócio
<code>id_produto</code>	<code>INT</code>	Não	PK	Identificador único do insumo/medicamento.
<code>nome</code>	<code>VARCHAR(100)</code>	Não	—	Nome descritivo comercial ou genérico do produto.
<code>unidade</code>	<code>VARCHAR(20)</code>	Não	—	Unidade de medida (ex: Frasco, Caixa, Ampola, Unidade).
<code>estoque</code>	<code>INT</code>	Não	—	Saldo físico atual em estoque (<code>DEFAULT 0</code>). Regra: <code>CHECK (estoque ≥ 0)</code> .
<code>estoque_minimo</code>	<code>INT</code>	Não	—	Cota de segurança para disparar alertas (<code>DEFAULT 0</code>). Regra: <code>≥ 0</code> .
<code>categoria_id</code>	<code>INT</code>	Não	FK	Referência para a categoria (<code>categoria.id_categoria</code>).

Tabela: `unidade_saude` (Destinos das Distribuições)

Campo	Tipo	Nulo	Chave	Descrição / Regra de Negócio
<code>id_unidade</code>	<code>INT</code>	Não	PK	Identificador único da unidade de saúde.
<code>nome</code>	<code>VARCHAR(100)</code>	Não	UK	Nome da unidade (ex: UBS Central, Policlínica, Secretaria de Saúde).
<code>endereco</code>	<code>VARCHAR(255)</code>	Sim	—	Endereço físico da unidade de atendimento.
<code>telefone</code>	<code>VARCHAR(20)</code>	Sim	—	Telefone de contato da unidade.
<code>ativo</code>	<code>BOOLEAN</code>	Não	—	Flag de controle lógico de ativação (<code>DEFAULT TRUE</code>).

Tabela: `movimentacao` (Cabeçalho de Entradas e Saídas)

Campo	Tipo	Nulo	Chave	Descrição / Regra de Negócio
id_movimentacao	INT	Não	PK	Identificador da movimentação em lote.
tipo	ENUM(...)	Não	—	Natureza da operação: 'Entrada' ou 'Saida'.
data_hora	DATETIME	Não	—	Data e hora exata da gravação (DEFAULT CURRENT_TIMESTAMP).
unidade_destino_id	INT	Sim	FK	Unidade recebedora (obrigatória na Saída, Secretaria na Entrada).
observacao	TEXT	Sim	—	Justificativa, número de nota fiscal ou anotação do operador.
usuario_id	INT	Não	FK	Operador responsável pelo lançamento (usuario.id_usuario).

Tabela: item_lancamento (Itens Pertencentes a uma Movimentação)

Campo	Tipo	Nulo	Chave	Descrição / Regra de Negócio
id_item	INT	Não	PK	Identificador único do item movimentado.
movimentacao_id	INT	Não	FK	Movimentação à qual o item pertence (ON DELETE CASCADE).
produto_id	INT	Não	FK	Insumo movimentado (ON DELETE RESTRICT).
quantidade	INT	Não	—	Quantidade lançada. Regra: CHECK (quantidade > 0) .

Tabela: Log (Trilha de Auditoria do Sistema)

Campo	Tipo	Nulo	Chave	Descrição / Regra de Negócio
id_log	INT	Não	PK	Identificador do evento de auditoria.
data_hora	DATETIME	Não	—	Timestamp do momento da ação (DEFAULT CURRENT_TIMESTAMP).
acao	VARCHAR(100)	Não	—	Nome da operação (ex: Login, Logout, Entrada, Saída, Exclusão).
descricao	TEXT	Sim	—	Detalhes auditáveis da ação (ex: quantidades, nomes e destinos).
usuario_id	INT	Não	FK	Usuário que executou a ação (usuario.id_usuario).

3.3. Integridade Referencial e Ações em Cascata (Foreign Keys)

Tabela de Origem	Chave Estrangeira	Tabela de Destino	Chave Referenciada	ON UPDATE	ON DELETE	Motivação da Regra
produto	categoria_id	categoria	id_categoria	CASCADE	RESTRICT	Impede apagar categoria que ainda possua produtos vinculados.
movimentacao	usuario_id	usuario	id_usuario	CASCADE	RESTRICT	Preserva o autor histórico da movimentação no almoxarifado.
movimentacao	unidade_destino_id	unidade_saude	id_unidade	CASCADE	RESTRICT	Impede deletar unidade que já recebeu remessas de materiais.
item_lancamento	movimentacao_id	movimentacao	id_movimentacao	CASCADE	CASCADE	Se a movimentação for cancelada, remove os itens vinculados.
item_lancamento	produto_id	produto	id_produto	CASCADE	RESTRICT	Impede deletar produto que já tenha histórico de movimentação.
log	usuario_id	usuario	id_usuario	CASCADE	RESTRICT	Garante a imutabilidade da trilha de auditoria para fins legais.

4. Explicação Detalhada das Páginas (pages/ e index.php)

index.php (Dashboard Principal)

- **O que faz:** Página inicial após o login.
- **Scripts incluídos:** script/sessao.php , script/conexao.php , script/funcoes_estoque.php , script/sidebar.php .
- **Lógica:** Chama verificarSessao() . Executa buscarAlertasEstoque(\$conn) para contabilizar produtos vazios (estoque = 0) e produtos abaixo do mínimo. Exibe cards de atalho para os módulos e o banner de resumo de pendências.

pages/Login.php (Autenticação)

- **O que faz:** Formulário de acesso ao sistema (Email e Senha).
- **Lógica:**
 - Recebe POST com email e senha .
 - Busca o usuário no banco por email (SELECT id_usuario, nome, email, senha, tipo FROM usuario WHERE email = ?).
 - Valida a senha usando password_verify(\$senha, \$usuario['senha']) .

- Se correto: inicializa as variáveis de sessão `$_SESSION['id_usuario']`, `$_SESSION['nome']`, `$_SESSION['email']`, `$_SESSION['tipo']`.
- Registra o log de acesso via `registrarLog($conn, 'Login', 'Usuário ... entrou no sistema.', $id)`.
- Redireciona para `index.php`.

pages/lancamentos.php (Entradas e Saídas — O coração do sistema)

- **O que faz:** Permite registrar entrada de insumos no almoxarifado ou saída de insumos para as UBS, com suporte a **múltiplos produtos por lançamento** (1 até 100+ produtos de uma vez).
- **Lógica de Backend (PHP no topo):**
 - Detecta se a requisição veio via AJAX (`isset($_POST['ajax'])`) ou cabeçalho `HTTP_X_REQUESTED_WITH`).
 - Se for Entrada: chama `lançamentoEntrada(...)`.
 - Se for Saída: chama `lançamentoSaída(...)`.
 - **Resposta:** Se for requisição AJAX, retorna JSON puro (`{"sucesso": true/false, "mensagem": "..."}`) e encerra com `exit`. Se for POST tradicional, redireciona gravando mensagem na sessão.
- **Lógica de Frontend (JavaScript embutido no rodapé):**
 - `alterarTipo()` : Ao alternar entre Entrada e Saída:
 - Se for **Entrada**: fixa o destino como Secretaria de Saúde e desabilita o campo (`disabled = true`).
 - Se for **Saída**: habilita o campo de destino e oculta a opção da própria Secretaria (obriga a mandar para uma UBS).
 - `adicionarProduto()` : Cria dinamicamente uma nova linha (`.produto-item`) com select do produto, input de quantidade, botão remover e div de avisos de estoque (`.produto-item_info`).
 - `removerProduto(botao)` : Remove a linha (impede remover se só houver uma).
 - `validarLinha(linhaDiv)` : Lê o atributo `data-estoque` do produto selecionado. Se a quantidade digitada for maior que o estoque, pinta a linha de vermelho (`.com-erro`) e mostra mensagem imediata:  Quantidade (X) excede o estoque disponível (Y).
 - `validarTodosProdutos()` : Além de validar cada linha, **soma as quantidades de produtos repetidos**. Se o usuário colocou o mesmo produto em 2 linhas diferentes e a soma das duas ultrapassar o estoque, bloqueia e avisa.
 - `formLancamento.addEventListener('submit', ...)` :
 - Intercepta o envio com `e.preventDefault()`.
 - Executa a validação. Se houver erro, cancela o envio, exibe o modal e rola a tela até a linha com erro.
 - Se válido, envia via `fetch()` em segundo plano com `FormData`.
 - **Se der erro no servidor:** Exibe o Modal Overlay com o erro. **Não recarrega a página**. Todos os 100 produtos preenchidos continuam na tela!
 - **Se der sucesso:** Exibe o Modal Overlay verde. Ao clicar em **"OK, Concluir"**, recarrega a página para limpar o formulário e carregar os estoques atualizados.

pages/produtos.php (Catálogo de Produtos)

- **O que faz:** Tabela com todos os produtos, quantidades, unidades de medida e estoques mínimos.
- **Script vinculado:** `script/produtos.js`.
- **Lógica:** A tabela é gerada via `listarProdutos($conn)`. No cliente, o `produtos.js` filtra as linhas em tempo real por nome (sem acento), categoria e unidade sem precisar recarregar a página.

pages/cadastrar_produto.php e pages/editar_produto.php

- **O que fazem:** Inclusão, edição e exclusão de itens no catálogo.
- **Lógica:**
 - Validações de duplicidade de nome.
 - Permite criar categorias dinamicamente pelo modal.
 - Ao salvar ou excluir, grava o log na tabela `log` (`Cadastro de produto`, `Edição de produto`, `Exclusão de produto`).
 - Na exclusão: se o produto já tiver sido movimentado no passado, o banco bloqueia a exclusão por Foreign Key (`RESTRICT`), e o código captura o erro amigavelmente.

pages/alertas.php (Painel de Alertas de Reposição)

- **O que faz:** Lista apenas os produtos que precisam de reposição urgente.
- **Lógica:** Usa `buscarAlertasEstoque($conn)`. Separa visualmente produtos com estoque zerado (crítico) e produtos abaixo da cota mínima configurada.

pages/estoque.php (Visão Geral de Saldos)

- **O que faz:** Exibe a lista completa de estoque com badges de situação (Normal, Baixo, Crítico).

pages/historico.php (Extrato de Movimentações)

- **O que faz:** Exibe tudo o que entrou e saiu do almoxarifado.
- **Lógica:** Faz `JOIN` entre as tabelas `movimentacao`, `item_lancamento`, `produto`, `unidade_saude` e `usuario`. Permite filtrar por tipo (Entrada/Saída), unidade de destino e período de datas.

pages/graficos.php (Indicadores Gerenciais)

- **O que faz:** Apresenta relatórios estatísticos visuais de movimentação.
- **Lógica:** Usa `script/funcoes_graficos.php` para agrupar totais por mês, por categoria e produtos mais demandados.

pages/usuarios.php, cadastrar_usuario.php e editar_usuario.php

- **O que fazem:** Gestão das contas que podem acessar o sistema.
- **Permissão:** Restrito exclusivamente a quem tem nível `Administrador` (`verificarTipo(['Administrador'])`).
- **Lógica:**
 - Senhas são criptografadas com `password_hash($senha, PASSWORD_DEFAULT)`.
 - Na tabela de listagem, a coluna de senha exibe texto fixo mascarado (`.....`).
 - Regra de segurança: um administrador **não pode excluir sua própria conta** conectada.
 - Cada operação gera log na auditoria.

pages/logs.php (Trilha de Auditoria)

- **O que faz:** Consulta o histórico de tudo o que foi feito no sistema.
- **Permissão:** Restrito a `Administrador`.
- **Lógica:** Formulário com filtros de **Usuário**, **Data Início** e **Data Fim**. Executa `buscarLogs(...)` com limite de segurança de 300 registros mais recentes.

pages/perfil.php (Dados do Usuário Conectado)

- **O que faz:** Permite que qualquer usuário logado altere seu próprio nome, email ou troque de senha.

5. Explicação Detalhada dos Scripts (script/)

script/conexao.php

- Cria a instância global `$conn = new mysqli($host, $usuario, $senha, $banco)`.
- Se falhar, interrompe a execução com mensagem de erro.
- O charset padrão utilizado é UTF-8.

script/sessao.php

- `session_start()`: Inicia ou retoma a sessão PHP.
- `define('BASE_URL', '/git/ESTAGIO/')`: Define o prefixo global de caminhos absolutos do sistema.
- `verificarSessao()`: Se não houver `$_SESSION['id_usuario']`, manda para a tela de login.
- `verificarTipo($tiposPermitidos)`: Recebe um array (ex: `['Administrador']`). Se o usuário não tiver o tipo correspondente, devolve HTTP 403 e encerra o script.

script/funcoes_lancamentos.php

Possui as duas funções mais importantes do sistema:

1. `lancamentoEntrada($conn, $produtos, $unidadeDestino, $observacao, $usuario_id)` :

- Inicia transação: `$conn->begin_transaction()` .
- Insere na tabela `movimentacao` (`tipo = 'Entrada'`).
- Itera pelo array de produtos: insere cada um em `item_lancamento` e faz `UPDATE produto SET estoque = estoque + ? WHERE id_produto = ?` .
- Dispara `registrarLog()` para cada produto inserido.
- Executa `$conn->commit()` .
- Se ocorrer qualquer exceção: executa `$conn->rollback()` e retorna `['sucesso' => false, 'mensagem' => $e->getMessage()]` .

2. `lancamentoSaida($conn, $produtos, $unidadeDestino, $observacao, $usuario_id)` :

- Inicia transação: `$conn->begin_transaction()` .
- Valida se a Unidade de Saúde de destino existe.
- Insere na tabela `movimentacao` (`tipo = 'Saida'`).
- Itera pelos produtos:
- Consulta o estoque atual no banco: `SELECT estoque, nome FROM produto WHERE id_produto = ?` .
- **Checagem atômica** : Se `quantidade > estoque` , interrompe imediatamente disparando uma `Exception` detalhada: `"Estoque insuficiente para o produto 'Nome'. Disponível: X, Solicitado: Y."`
- Atualiza com trava: `UPDATE produto SET estoque = estoque - ? WHERE id_produto = ? AND estoque >= ?` .
- Valida se `affected_rows > 0` .
- Insere em `item_lancamento` .
- Busca o nome da unidade de destino e registra o log formatado: `"Saída de X [unidade] de [Produto] para [Nome da Unidade]."`
- Executa `$conn->commit()` .
- Se falhar: `$conn->rollback()` desfaz tudo e retorna array de erro.

script/funcoes_logs.php

- `registrarLog($conn, $acao, $descricao, $usuario_id)` :
- Prepara `INSERT INTO log (acao, descricao, usuario_id) VALUES (?, ?, ?)` .
- Executa via Prepared Statement seguro.
- `buscarLogs($conn, $usuario, $dataInicio, $dataFim)` :
- Monta SQL dinâmica com base nos filtros preenchidos.
- Faz bind dos tipos e parâmetros dinamicamente.
- Ordena por `data_hora DESC, id_log DESC` com `LIMIT 300` .

script/funcoes_produtos.php

- `buscarCategorias($conn)` : Retorna lista de categorias para popular selects.
- `buscarUnidades($conn)` : Retorna unidades de medida distintas já cadastradas.
- `criarCategoria($conn, $nome)` : Cria nova categoria com checagem de duplicidade.
- `salvarProduto($conn, $dados)` : Insere ou atualiza produtos no banco com Prepared Statements.
- `listarProdutos($conn)` : Gera as linhas `<tr>` da tabela de produtos com os data-attributes necessários para o filtro em JavaScript.

script/funcoes_usuarios.php

- `cadastrarUsuario($conn, $nome, $email, $senha, $tipo)` : Hasheia a senha com `PASSWORD_DEFAULT` e cadastra no banco.
- `atualizarUsuario(...)` : Atualiza dados do usuário. Se o campo de senha vier preenchido, gera um novo hash; se vier vazio, preserva a senha antiga.
- `excluirUsuario($conn, $id)` : Deleta o usuário.
- `listarUsuarios($conn)` : Gera as linhas da tabela de usuários.

script/funcoes_estoque.php

- `buscarAlertasEstoque($conn)` : Executa query que busca produtos onde `estoque ≤ estoque_minimo` , ordenando pelos casos mais críticos (zerados primeiro).

script/sidebar.php

- `sidebar($paginaAtiva)` : Função que renderiza o menu lateral de navegação em todas as páginas, marcando visualmente o link ativo de acordo com o parâmetro `$paginaAtiva` .

script/produtos.js

- Script de frontend client-side puro. Ouve os eventos `input` da barra de pesquisa e `change` dos filtros de categoria e unidade de medida.
- Normaliza termos de busca removendo acentos (`.normalize('NFD').replace(/[\u0300-\u036f]/g, '')`) para que "remedio" encontre "remédio".
- Oculta ou exibe as linhas `<tr>` da tabela alterando a propriedade `.hidden` .

script/logout.php

- Registra o evento de saída na tabela `log` .
- Limpa o array `$_SESSION` e executa `session_destroy()` .
- Redireciona para a tela de login.

6. Particularidades Técnicas, Casos Especiais e Decisões de Arquitetura

Para o técnico de TI que precisar dar manutenção, esta seção documenta os pontos de atenção, soluções de contorno e tratamentos específicos adotados no código:

1. Injeção Manual de Campo `disabled` via JavaScript (`pages/lancamentos.php`)

- **O Problema:** No HTML padrão, quando um elemento de formulário tem o atributo `disabled` (como `<select id="unidade_destino" disabled>`), o navegador **omite esse campo** ao criar um objeto `new FormData(form)` . Nas entradas de material, a unidade de destino é fixada na Secretaria de Saúde e fica `disabled` . Logo, o PHP não recebia o ID da unidade de destino.
- **Solução Adotada:** No JavaScript que intercepta o submit, forçamos a inserção manual do campo antes de enviar o `fetch` :

```
javascript const formData = new FormData(formLancamento); const selectUnidade = document.getElementById('unidade_destino'); if (selectUnidade && selectUnidade.value) { formData.set('unidade_destino', selectUnidade.value); // Injeta mesmo se estiver disabled! } formData.append('ajax', '1');
```

2. Loop PHP embutido dentro de String JavaScript (`pages/lancamentos.php`)

- **O Problema:** Ao clicar em "+ Adicionar produto", o JavaScript precisa criar um novo `<select>` contendo todos os produtos do banco e o estoque de cada um no atributo `data-estoque` .
- **Como foi feito:** Como o script está dentro do próprio arquivo `.php` , o laço `while` do PHP foi colocado diretamente dentro da string template do JavaScript:

```
javascript div.innerHTML = ` <select name="produtos[${contadorProdutos}][produto_id]" class="select-produto" required> <option value="">Selecione um produto</option> <?php $resultadoProdutos = $conn->query($sqlProdutos); while ($produto = $resultadoProdutos->fetch_assoc()): ?> <option value="<?=$produto['id_produto'] ?>" data-estoque="<?=$produto['estoque'] ?>"> <?=$htmlspecialchars($produto['nome']) ?> - Estoque: <?=$produto['estoque'] ?> </option> <?php endwhile; ?> </select> ... `;
```
- **Atenção para o TI:** Se você quiser mover esse código JS para um arquivo `.js` externo, o PHP **não será executado**. Para externalizar, você precisará usar uma tag `<template>` no HTML ou clonar o primeiro `<select>` com `cloneNode(true)` .

3. A Constante `BASE_URL` (`script/sessao.php`)

- **O Problema:** Os redirecionamentos do PHP (`header('Location: ...')`) precisam de caminhos absolutos para não se perderem dependendo de qual subpasta a página está.

- **Como foi feito:** Foi criada a constante `define('BASE_URL', '/git/ESTAGIO/');`.
- **Atenção para o TI:** Se você renomear a pasta do projeto (por exemplo, para `http://localhost/gestsaudef/` ou se colocar na raiz de um VirtualHost `http://estoque.saude.gov.br/`), **VOCÊ DEVE ALTERAR ESSA LINHA** em `script/sessao.php` para `/gestsaude/` ou `/`. Senão, todas as páginas darão erro 404 ao redirecionar.

4. Concatenação de Horário nos Filtros de Data (`script/funcões_logs.php`)

- **O Problema:** O campo `data_hora` da tabela `log` é do tipo `DATETIME` (`2026-09-05 14:32:10`). Quando o operador filtrava logs de "01/09/2026" até "05/09/2026", o input `date` do HTML mandava apenas `'2026-09-05'`. O MySQL comparava com `'2026-09-05 00:00:00'`, e com isso todos os logs acontecidos durante o próprio dia 05 eram ignorados.
- **Solução Adotada:** No PHP, concatenamos `" 23:59:59"` na data final:

```
php if ($dataFim == '') { $sql .= " AND l.data_hora <= ?"; $parametros[] = $dataFim . " 23:59:59"; $tipos .= "s"; }
```

5. Preservação de Formulário via Fetch sem Reload (`pages/Lancamentos.php`)

- **O Problema:** O operador preenchia 50 ou 100 produtos para enviar para uma UBS. Se o produto nº 48 estivesse com quantidade maior do que o estoque, o sistema anterior recarregava a página inteira, e o operador **perdia tudo o que tinha digitado**, precisando redigitar os 50 produtos do zero.
- **A Solução:**
- O envio do formulário foi transformado em assíncrono com `fetch()` e cabeçalho `X-Requested-With: XMLHttpRequest`.
- O PHP detecta requisição AJAX e responde em JSON.
- Se houver erro, a página **não recarrega**. O JavaScript exibe o modal de erro e mantém todos os inputs e selects exatamente como o usuário digitou, permitindo corrigir só a linha errada e tentar de novo.

6. Modal Overlay com Recarregamento Condicional no "OK"

- **O Problema:** Quando o lançamento era um sucesso, se a página não recarregasse, os valores continuavam no formulário e o operador corria o risco de clicar de novo e duplicar o envio. Por outro lado, se a página recarregasse muito rápido, o operador não conseguia ler a mensagem verde de sucesso.
- **A Solução:** Criamos o Modal Overlay (`.modal-overly`) em tela cheia com efeito de desfoque de fundo (`backdrop-filter: blur(4px)`).
- **No Sucesso:** O modal exibe um botão "OK, Concluir". Ele fica parado na tela o tempo que o usuário quiser. Ao clicar no botão, a função `fecharMensagem(true)` executa `window.location.href = 'lancamentos.php'`, recarregando a tela limpa e com os estoques novos já abatidos do banco.
- **No Erro:** O botão do modal chama `fecharMensagem(false)`. Ele fecha o modal e não mexe na página, mantendo os 100 produtos na tela.

7. Validação Cumulativa de Linhas Repetidas no JavaScript

- **O Problema:** Se o estoque do produto X é 10, o operador podia burlar a checagem colocando o produto X na linha 1 com quantidade 6 e o produto X na linha 2 com quantidade 7 (total 13 > 10). Individualmente nenhuma linha passava de 10, mas a soma estourava o estoque.
- **A Solução:** Em `validarTodosProdutos()`, o JS cria um mapa acumulador `somaPorProduto[prodId]`. Ele soma as quantidades de todas as linhas que têm o mesmo produto. Se a soma exceder o estoque, todas as linhas daquele produto são pintadas de vermelho com aviso de soma excedida.

8. Bloqueio de Rolagem do Mouse nos Inputs Numéricos

- **O Problema:** Quando o operador usa o scroll do mouse para descer a página longa de lançamentos, se o cursor do mouse passar por cima de um `<input type="number">`, o navegador por padrão incrementa ou decrementa o número sem o usuário perceber.
- **Solução Adotada:** Foi adicionado um listener global no JavaScript de lançamentos:

```
javascript document.addEventListener('wheel', function(evento) { if (evento.target.matches('input[type="number"]')) { evento.target.blur(); // Tira o foco do campo na hora do scroll } });
```

9. Bloqueio de Auto-Exclusão do Administrador (`pages/usuarios.php`)

- **O Problema:** Se o único administrador logado clicasse para excluir o seu próprio usuário, ele ficaria sem acesso ao sistema e ninguém mais conseguiria gerenciar os acessos.
- **Solução Adotada:** No backend de `usuarios.php`, antes de executar a exclusão, verifica-se:

```
php if ((int)$idUsuarioExcluir === (int)$_SESSION['id_usuario']) { // Bloqueia com mensagem: "Você não pode excluir o seu próprio usuário conectado." }
```

10. Requisito Inegociável da Engine InnoDB no MySQL

- **Atenção Crítica:** O sistema depende de transações `$conn→begin_transaction()`, `$conn→commit()` e `$conn→rollback()` para não deixar lançamentos gravados pela metade se ocorrer erro no meio do processo.
- **Atenção para o TI:** O banco **TEM que ser InnoDB**. Se o banco for exportado e restaurado em um servidor antigo com engine padrão **MyISAM**, transações **NÃO FUNCIONAM** (o MySQL não dá erro, mas o `rollback()` é simplesmente ignorado e não desfaz os dados).